# Chapter 5. Money

This chapter examines how to bring funding into a free software environment. It is aimed not only at developers who are paid to work on free software projects, but also at their managers, who need to understand the social dynamics of the development environment. In the sections that follow, the addressee ("you") is presumed to be either a paid developer, or one who manages such developers. The advice will often be the same for both; when it's not, the intended audience will be made clear from context.

Corporate funding of free software development is not a new phenomenon. A lot of development has always been informally subsidized. When a system administrator writes a network analysis tool to help her do her job, then posts it online and gets bug fixes and feature contributions from other system administrators, what's happened is that an unofficial consortium has been formed. The consortium's funding comes from the sysadmins' salaries, and its office space and network bandwidth are donated, albeit unknowingly, by the organizations they work for. Those organizations benefit from the investment, of course, although they may not be institutionally aware of it at first.

The difference today is that many of these efforts are being formalized. Corporations have become conscious of the benefits of open source software, and started involving themselves more directly in its development. Developers too have come to expect that really important projects will attract at least donations, and possibly even long-term sponsors. While the presence of money has not changed the basic dynamics of free software development, it has greatly changed the scale at which things happen, both in terms of the number of developers and time-per-developer. It has also had effects on how projects are organized, and on how the parties involved in them interact. The issues are not merely about how the money is spent, or how return on investment is measured. They are also about management and process: how can the hierarchical command structures of corporations and the semi-decentralized volunteer communities of free software projects work productively with each other? Will they even agree on what "productively" means?

Financial backing is, in general, welcomed by open source development communities. It can reduce a project's vulnerability to the Forces of Chaos, which sweep away so many projects before they really get off the ground, and therefore it can make people more willing to give the software a chance—they feel they're investing their time into something that will still be around six months from now. After all, credibility is contagious, to a point. When, say, IBM backs an open source project, people pretty much assume the project won't be allowed to fail, and their resultant willingness to devote effort to it can make that a self-fulfilling prophecy.

However, funding also brings a perception of control. If not handled carefully, money can divide a project into in-group and out-group developers. If the unpaid volunteers get the feeling that design decisions or feature additions are simply available to the highest bidder, they'll head off to a project that seems more like a meritocracy and less like unpaid labor for someone else's benefit. They may never complain overtly on the mailing lists. Instead, there will simply be less and less noise from external sources, as the volunteers gradually stop trying to be taken seriously. The buzz of small-scale activity will continue, in the form of bug reports and occasional small fixes. But there won't be any large code contributions or outside participation in design discussions. People sense what's expected of them, and live up (or down) to those expectations.

Although money needs to be used carefully, that doesn't mean it can't buy influence. It most certainly can. The trick is that it can't buy influence directly. In a straightforward commercial transaction, you trade money for what you want. If you need a feature added, you sign a contract, pay for it, and it gets done. In an open source project, it's not so simple. You may sign a contract with some developers, but they'd be fooling themselves—and you—if they guaranteed that the work you paid for would be accepted by the development community simply because you paid for it. The work can only be accepted on its own merits and on how it fits into the community's vision for the software. You may have some say in that vision, but you won't be the only voice.

So money can't purchase influence, but it can purchase things that *lead to* influence. The most obvious example is programmers. If good programmers are hired, and they stick around long enough to get ex-

perience with the software and credibility in the community, then they can influence the project by the same means as any other member. They will have a vote, or if there are many of them, they will have a voting bloc. If they are respected in the project, they will have influence beyond just their votes. There is no need for paid developers to disguise their motives, either. After all, everyone who wants a change made to the software wants it for a reason. Your company's reasons are no less legitimate than anyone else's. It's just that the weight given to your company's goals will be determined by its representatives' status in the project, not by the company's size, budget, or business plan.

# Types of Involvement

There are many different reasons open source projects get funded. The items in this list aren't mutually exclusive; often a project's financial backing will result from several, or even all, of these motivations:

Sharing the burden
: Separate organizations with related software needs often find themselves duplicating effort, either by redundantly writing similar code in-house, or by purchasing similar products from proprietary vendors. When they realize what's going on, the organizations may pool their resources and create (or join) an open source project tailored to their needs. The advantages are obvious: the costs of development are divided, but the benefits accrue to all. Although this scenario seems most intuitive for nonprofits, it can make strategic sense even for for-profit competitors.

  Examples: http://www.openadapter.org/, http://www.koha.org/

Augmenting services
: When a company sells services which depend on, or are made more attractive by, particular open source programs, it is naturally in that company's interests to ensure those programs are actively maintained.

  Example: CollabNet's [http://www.collab.net/] support of http://subversion.tigris.org/ (disclaimer: that's my day job, but it's also a perfect example of this model).

Supporting hardware sales
: The value of computers and computer components is directly related to the amount of software available for them. Hardware vendors—not just whole-machine vendors, but also makers of peripheral devices and microchips—have found that having high-quality free software to run on their hardware is important to customers.

Undermining a competitor
: Sometimes companies support a particular open source project as a means of undermining a competitor's product, which may or may not be open source itself. Eating away at a competitor's market share is usually not the sole reason for getting involved with an open source project, but it can be a factor.

  Example: http://www.openoffice.org/ (no, this isn't the only reason OpenOffice exists, but the software is at least partly a response to Microsoft Office).

Marketing
: Having your company associated with a popular open source application can be simply good brand management.

Dual-licensing
: *Dual-licensing* is the practice of offering software under a traditional proprietary license for customers who want to resell it as part of a proprietary application of their own, and simultaneously under a free license for those willing to use it under open source terms (see the section called “Dual Licensing Schemes” in Chapter 9, *Licenses, Copyrights, and Patents*). If the open source developer community is active, the software gets the benefits of wide-area debugging and development, yet the company still gets a royalty stream to support some full-time programmers.

Two well-known examples are MySQL [http://www.mysql.com/], makers of the database software of the same name, and Sleepycat [http://www.sleepycat.com/], which offers distributions and support for the Berkeley Database. It's no coincidence that they're both database companies. Database software tends to be integrated into applications rather than marketed directly to users, so it's very well-suited to the dual-licensing model.

Donations
: A widely-used project can sometimes get significant contributions, from both individuals and organizations, just by having an online donation button, or sometimes by selling branded merchandise such as coffee mugs, T-shirts, mousepads, etc. A word of caution: if your project accepts donations, plan out how the money will be used *before* it comes in, and state the plans on the project's web site. Discussions about how to allocate money tend to go a lot more smoothly when held before there's actual money to spend; and anyway, if there are significant disagreements, it's better to find that out while it's still academic.

A funder's business model is not the only factor in how it relates to an open source community. The historical relationship between the two also matters: did the company start the project, or is it joining an existing development effort? In both cases, the funder will have to earn credibility, but, not surprisingly, there's a bit more earning to be done in the latter case. The organization needs to have clear goals with respect to the project. Is the company trying to keep a position of leadership, or simply trying to be one voice in the community, to guide but not necessarily govern the project's direction? Or does it just want to have a couple of committers around, able to fix customers' bugs and get the changes into the public distribution without any fuss?

Keep these questions in mind as you read the guidelines that follow. They are meant to apply to any sort of organizational involvement in a free software project, but every project is a human environment, and therefore no two are exactly alike. To some degree, you will always have to play by ear, but following these principles will increase the likelihood of things turning out the way you want.

# Hire for the Long Term

If you're managing programmers on an open source project, keep them there long enough that they acquire both technical and political expertise—a couple of years, at a minimum. Of course, no project, whether open or closed-source, benefits from swapping programmers in and out too often. The need for a newcomer to learn the ropes each time would be a deterrent in any environment. But the penalty is even stronger in open source projects, because outgoing developers take with them not only their knowledge of the code, but also their status in the community and the human relationships they have made there.

The credibility a developer has accumulated cannot be transferred. To pick the most obvious example, an incoming developer can't inherit commit access from an outgoing one (see the section called “Money Can't Buy You Love” later in this chapter), so if the new developer doesn't already have commit access, he will have to submit patches until he does. But commit access is only the most measurable manifestation of lost influence. A long-time developer also knows all the old arguments that have been hashed and rehashed on the discussion lists. A new developer, having no memory of those conversations, may try to raise the topics again, leading to a loss of credibility for your organization; the others might wonder "Can't they remember anything?" A new developer will also have no political feel for the project's personalities, and will not be able to influence development directions as quickly or as smoothly as one who's been around a long time.

Train newcomers through a program of supervised engagement. The new developer should be in direct contact with the public development community from the very first day, starting off with bug fixes and cleanup tasks, so he can learn the code base and acquire a reputation in the community, yet not spark any long and involved design discussions. All the while, one or more experienced developers should be available for questioning, and should be reading every post the newcomer makes to the development lists, even if they're in threads that the experienced developers normally wouldn't pay attention to. This

will help the group spot potential rocks before the newcomer runs aground. Private, behind-the-scenes encouragement and pointers can also help a lot, especially if the newcomer is not accustomed to massively parallel peer review of his code.

When CollabNet hires a new developer to work on Subversion, we sit down together and pick some open bugs for the new person to cut his teeth on. We'll discuss the technical outlines of the solutions, and then assign at least one experienced developer to (publicly) review the patch that the new developer will (also publicly) post. We typically don't even look at the patch before the main development list sees it, although we could if there were some reason to. The important thing is that the new developer go through the process of public review, learning the code base while simultaneously becoming accustomed to receiving critiques from complete strangers. But we try to coordinate the timing so that our own review comes immediately after the posting of the patch. That way the first review the list sees is ours, which can help set the tone for the others' reviews. It also contributes to the idea that this new person is to be taken seriously: if others see that we're putting in the time to give detailed reviews, with thorough explanations and references into the archives where appropriate, they'll appreciate that a form of training is going on, and that it probably signifies a long-term investment. This can make them more positively disposed toward that developer, at least to the degree of spending a little extra time answering questions and reviewing patches.

# Appear as Many, Not as One

Your developers should strive to appear in the project's public forums as individual participants, rather than as a monolithic corporate presence. This is not because there is some negative connotation inherent in monolithic corporate presences (well, perhaps there is, but that's not what this book is about). Rather, it's because individuals are the only sort of entity open source projects are structurally equipped to deal with. An individual contributor can have discussions, submit patches, acquire credibility, vote, and so forth. A company cannot.

Furthermore, by behaving in a decentralized manner, you avoid stimulating centralization of opposition. Let your developers disagree with each other on the mailing lists. Encourage them to review each other's code as often, and as publicly, as they would anyone else's. Discourage them from always voting as a bloc, because if they do, others may start to feel that, just on general principles, there should be an organized effort to keep them in check.

There's a difference between actually being decentralized and simply striving to appear that way. Under certain circumstances, having your developers behave in concert can be quite useful, and they should be prepared to coordinate behind the scenes when necessary. For example, when making a proposal, having several people chime in with agreement early on can help it along, by giving the impression of a growing consensus. Others will feel that the proposal has momentum, and that if they were to object, they'd be stopping that momentum. Thus, people will object only if they have a good reason to do so. There's nothing wrong with orchestrating agreement like this, as long as objections are still taken seriously. The public manifestations of a private agreement are no less sincere for having been coordinated beforehand, and are not harmful as long as they are not used to prejudicially snuff out opposing arguments. Their purpose is merely to inhibit the sort of people who like to object just to stay in shape; see the section called “The Softer the Topic, the Longer the Debate” in Chapter 6, *Communications* for more about them.

# Be Open About Your Motivations

Be as open about your organization's goals as you can without compromising business secrets. If you want the project to acquire a certain feature because, say, your customers have been clamoring for it, just say so outright on the mailing lists. If the customers wish to remain anonymous, as is sometimes the case, then at least ask them if they can be used as unnamed examples. The more the public development community knows about *why* you want what you want, the more comfortable they'll be with whatever you're proposing.

This runs counter to the instinct—so easy to acquire, so hard to shake off—that knowledge is power, and that the more others know about your goals, the more control they have over you. But that instinct would be wrong here. By publicly advocating the feature (or bugfix, or whatever it is), you have *already* laid your cards on the table. The only question now is whether you will succeed in guiding the community to share your goal. If you merely state that you want it, but can't provide concrete examples of why, your argument is weak, and people will start to suspect a hidden agenda. But if you give just a few real-world scenarios showing why the proposed feature is important, that can have a dramatic effect on the debate.

To see why this is so, consider the alternative. Too frequently, debates about new features or new directions are long and tiresome. The arguments people advance often reduce to "I personally want X," or the ever-popular "In my years of experience as a software designer, X is extremely important to users / a useless frill that will please no one." Predictably, the absence of real-world usage data neither shortens nor tempers such debates, but instead allows them to drift farther and farther from any mooring in actual user experience. Without some countervailing force, the end result is as likely as not to be determined by whoever was the most articulate, or the most persistent, or the most senior.

As an organization with plentiful customer data available, you have the opportunity to provide just such a countervailing force. You can be a conduit for information that might otherwise have no means of reaching the development community. The fact that that information supports your desires is nothing to be embarrassed about. Most developers don't individually have very broad experience with how the software they write is used. Each developer uses the software in her own idiosyncratic way; as far as other usage patterns go, she's relying on intuition and guesswork, and deep down, she knows this. By providing credible data about a significant number of users, you are giving the public development community something akin to oxygen. As long as you present it right, they will welcome it enthusiastically, and it will propel things in the direction you want to go.

The key, of course, is presenting it right. It will never do to insist that simply because you deal with a large number of users, and because they need (or think they need) a given feature, therefore your solution ought to be implemented. Instead, you should focus your initial posts on the problem, rather than on one particular solution. Describe in great detail the experiences your customers are encountering, offer as much analysis as you have available, and as many reasonable solutions as you can think of. When people start speculating about the effectiveness of various solutions, you can continue to draw on your data to support or refute what they say. You may have one particular solution in mind all along, but don't single it out for special consideration at first. This is not deception, it is simply standard "honest broker" behavior. After all, your true goal is to solve the problem; a solution is merely a means to that end. If the solution you prefer really is superior, other developers will recognize that on their own eventually—and then they will get behind it of their own free will, which is much better than you browbeating them into implementing it. (There is also the possibility that they will think of a better solution.)

This is not to say that you can't ever come out in favor of a specific solution. But you must have the patience to see the analysis you've already done internally repeated on the public development lists. Don't post saying "Yes, we've been over all that here, but it doesn't work for reasons A, B, and C. When you get right down to it, the only way to solve this is..." The problem is not so much that it sounds arrogant as that it gives the impression that you have *already* devoted some unknown (but, people will presume, large) amount of analytical resources to the problem, behind closed doors. It makes it seem as though efforts have been going on, and perhaps decisions made, that the public is not privy to, and that is a recipe for resentment.

Naturally, *you* know how much effort you've devoted to the problem internally, and that knowledge is, in a way, a disadvantage. It puts your developers in a slightly different mental space than everyone else on the mailing lists, reducing their ability to see things from the point of view of those who haven't yet thought about the problem as much. The earlier you can get everyone else thinking about things in the same terms as you do, the smaller this distancing effect will be. This logic applies not only to individual technical situations, but to the broader mandate of making your goals as clear as you can. The unknown is always more destabilizing than the known. If people understand why you want what you want, they'll feel comfortable talking to you even when they disagree. If they can't figure out what makes you tick, they'll assume the worst, at least some of the time.

You won't be able to publicize everything, of course, and people won't expect you to. All organizations have secrets; perhaps for-profits have more of them, but nonprofits have them too. If you must advocate a certain course, but can't reveal anything about why, then simply offer the best arguments you can under that handicap, and accept the fact that you may not have as much influence as you want in the discussion. This is one of the compromises you make in order to have a development community not on your payroll.

# Money Can't Buy You Love

If you're a paid developer on a project, then set guidelines early on about what the money can and cannot buy. This does not mean you need to post twice a day to the mailing lists reiterating your noble and incorruptible nature. It merely means that you should be on the lookout for opportunities to defuse the tensions that *could* be created by money. You don't need to start out assuming that the tensions are there; you do need to demonstrate an awareness that they have the potential to arise.

A perfect example of this came up in the Subversion project. Subversion was started in 2000 by CollabNet [http://www.collab.net/], which has been the project's primary funder since its inception, paying the salaries of several developers (disclaimer: I'm one of them). Soon after the project began, we hired another developer, Mike Pilato, to join the effort. By then, coding had already started. Although Subversion was still very much in the early stages, it already had a development community with a set of basic ground rules.

Mike's arrival raised an interesting question. Subversion already had a policy about how a new developer gets commit access. First, he submits some patches to the development mailing list. After enough patches have gone by for the other committers to see that the new contributor knows what he's doing, someone proposes that he just commit directly (that proposal is private, as described in the section called “Committers”). Assuming the committers agree, one of them mails the new developer and offers him direct commit access to the project's repository.

CollabNet had hired Mike specifically to work on Subversion. Among those who already knew him, there was no doubt about his coding skills or his readiness to work on the project. Furthermore, the volunteer developers had a very good relationship with the CollabNet employees, and most likely would not have objected if we'd just given Mike commit access the day he was hired. But we knew we'd be setting a precedent. If we granted Mike commit access by fiat, we'd be saying that CollabNet had the right to ignore project guidelines, simply because it was the primary funder. While the damage from this would not necessarily be immediately apparent, it would gradually result in the non-salaried developers feeling disenfranchised. Other people have to earn their commit access—CollabNet just buys it.

So Mike agreed to start out his employment at CollabNet like any other volunteer developer, without commit access. He sent patches to the public mailing list, where they could be, and were, reviewed by everyone. We also said on the list that we were doing things this way deliberately, so there could be no missing the point. After a couple of weeks of solid activity by Mike, someone (I can't remember if it was a CollabNet developer or not) proposed him for commit access, and he was accepted, as we knew he would be.

That kind of consistency gets you a credibility that money could never buy. And credibility is a valuable currency to have in technical discussions: it's immunization against having one's motives questioned later. In the heat of argument, people will sometimes look for non-technical ways to win the battle. The project's primary funder, because of its deep involvement and obvious concern over the directions the project takes, presents a wider target than most. By being scrupulous to observe all project guidelines right from the start, the funder makes itself the same size as everyone else.

(See also Danese Cooper's blog at http://blogs.sun.com/roller/page/DaneseCooper/20040916 for a similar story about commit access. Cooper was then Sun Microsystem's "Open Source Diva"—I believe that was her official title—and in the blog entry, she describes how the Tomcat development community got Sun to hold its own developers to the same commit-access standards as the non-Sun developers.)

The need for the funders to play by the same rules as everyone else means that the Benevolent Dictatorship governance model (see the section called “Benevolent Dictators” in Chapter 4, *Social and Political Infrastructure*) is slightly harder to pull off in the presence of funding, particularly if the dictator works for the primary funder. Since a dictatorship has few rules, it is hard for the funder to prove that it's abiding by community standards, even when it is. It's certainly not impossible; it just requires a project leader who is able to see things from the point of view of the outside developers, as well as that of the funder, and act accordingly. Even then, it's probably a good idea to have a proposal for non-dictatorial governance sitting in your back pocket, ready to be brought out the moment there are any indications of widespread dissatisfaction in the community.

# Contracting

Contracted work needs to be handled carefully in free software projects. Ideally, you want a contractor's work to be accepted by the community and folded into the public distribution. In theory, it wouldn't matter who the contractor is, as long as his work is good and meets the project's guidelines. Theory and practice can sometimes match, too: a complete stranger who shows up with a good patch *will* generally be able to get it into the software. The trouble is, it's very hard to produce a good patch for a non-trivial enhancement or new feature while truly being a complete stranger; one must first discuss it with the rest of the project. The duration of that discussion cannot be precisely predicted. If the contractor is paid by the hour, you may end up paying more than you expected; if he is paid a flat sum, he may end up doing more work than he can afford.

There are two ways around this. The preferred way is to make an educated guess about the length of the discussion process, based on past experience, add in some padding for error, and base the contract on that. It also helps to divide the problem into as many small, independent chunks as possible, to increase the predictability of each chunk. The other way is to contract solely for delivery of a patch, and treat the patch's acceptance into the public project as a separate matter. Then it becomes much easier to write the contract, but you're stuck with the burden of maintaining a private patch for as long as you depend on the software, or at least for as long as it takes you to get that patch or equivalent functionality into the mainline. Of course, even with the preferred way, the contract itself cannot require that the patch be accepted into the code, because that would involve selling something that's not for sale. (What if the rest of the project unexpectedly decides not to support the feature?) However, the contract can require a *bona fide* effort to get the change accepted by the community, and that it be committed to the repository if the community agrees with it. For example, if the project has written standards regarding code changes, the contract can reference those standards and specify that the work must meet them. In practice, this usually works out the way everyone hopes.

The best tactic for successful contracting is to hire one of the project's developers—preferably a committer—as the contractor. This may seem like a form of purchasing influence, and, well, it is. But it's not as corrupt as it might seem. A developer's influence in the project is due mainly to the quality of his code and to his interactions with other developers. The fact that he has a contract to get certain things done doesn't raise his status in any way, and doesn't lower it either, though it may make people scrutinize him more carefully. Most developers would not risk their long-term position in the project by backing an inappropriate or widely disliked new feature. In fact, part of what you get, or should get, when you hire such a contractor is advice about what sorts of changes are likely to be accepted by the community. You also get a slight shift in the project's priorities. Because prioritization is just a matter of who has time to work on what, when you pay for someone's time, you cause their work to move up in the priority queue a bit. This is a well-understood fact of life among experienced open source developers, and at least some of them will devote attention to the contractor's work simply because it looks like it's going to *get done*, so they want to help it get done right. Perhaps they won't write any of the code, but they'll still discuss the design and review the code, both of which can be very useful. For all these reasons, the contractor is best drawn from the ranks of those already involved with the project.

This immediately raises two questions: Should contracts ever be private? And when they're not, should you worry about creating tensions in the community by the fact that you've contracted with some developers and not others?

It's best to be open about contracts, when you can. Otherwise, the contractor's behavior may seem strange to others in the community—perhaps he's suddenly giving inexplicably high priority to features he's never shown interest in in the past. When people ask him why he wants them now, how can he answer convincingly if he can't talk about the fact that he's been contracted to write them?

At the same time, neither you nor the contractor should act as though others should treat your arrangement as a big deal. Too often I've seen contractors waltz onto a development list with the attitude that their posts should be taken more seriously simply because they're being paid. That kind of attitude signals to the rest of the project that the contractor regards the fact of the contract—as opposed to the code *resulting* from the contract—to be the important thing. But from the other developers' point of view, only the code matters. At all times, the focus of attention should be kept on technical issues, not on the details of who is paying whom. For example, one of the developers in the Subversion community handles contracting in a particularly graceful way. While discussing his code changes in IRC, he'll mention as an aside (often in a private remark, an IRC *privmsg*, to one of the other committers) that he's being paid for his work on this particular bug or feature. But he also consistently gives the impression that he'd want to be working on that change anyway, and that he's happy the money is making it possible for him to do that. He may or may not reveal his customer's identity, but in any case he doesn't dwell on the contract. His remarks about it are just an ornament to an otherwise technical discussion about how to get something done.

That example shows another reason why it's good to be open about contracts. There may be multiple organizations sponsoring contracts on a given open source project, and if each knows what the others are trying to do, they may be able to pool their resources. In the above case, the project's largest funder (CollabNet) is not involved in any way with these piecework contracts, but knowing that someone else is sponsoring certain bug fixes allows CollabNet to redirect its resources to other bugs, resulting in greater efficiency for the project as a whole.

Will other developers resent that some are paid for working on the project? In general, no, particularly when those who are paid are established, well-respected members of the community anyway. No one expects contract work to be distributed equally among all the committers. People understand the importance of long-term relationships: the uncertainties involved in contracting are such that once you find someone you can work reliably with, you would be reluctant to switch to a different person just for the sake of evenhandedness. Think of it this way: the first time you hire, there will be no complaints, because clearly you had to pick *someone*—it's not your fault you can't hire everyone. Later, when you hire the same person a second time, that's just common sense: you already know him, the last time was successful, so why take unnecessary risks? Thus, it's perfectly natural to have one or two go-to people in the community, instead of spreading the work around evenly.

# Review and Acceptance of Changes

The community is still important to the success of contract work. Their involvement in the design and review process for sizeable changes cannot be an afterthought. It must be considered part of the work, and fully embraced by the contractor. Don't think of community scrutiny as an obstacle to be overcome—think of it as a free design board and QA department. It is a benefit to be aggressively pursued, not merely endured.

## Case study: the CVS password-authentication protocol

In 1995, I was one half of a partnership that provided support and enhancements for CVS (the Concurrent Versions System; see http://www.cvshome.org/). My partner Jim and I were, informally, the maintainers of CVS by that point. But we'd never thought carefully about how we ought to relate to the existing, mostly volunteer CVS development community. We just assumed that they'd send in patches, and we'd apply them, and that was pretty much how it worked.

Back then, networked CVS could be done only over a remote login program such as `rsh`. Using the same password for CVS access as for login access was an obvious security risk, and many organizations were put off by it. A major investment bank hired us to add a new authentication mechanism, so they

could safely use networked CVS with their remote offices.

Jim and I took the contract and sat down to design the new authentication system. What we came up with was pretty simple (the United States had export controls on cryptographic code at the time, so the customer understood that we couldn't implement strong authentication), but as we were not experienced in designing such protocols, we still made a few gaffes that would have been obvious to an expert. These mistakes would easily have been caught had we taken the time to write up a proposal and run it by the other developers for review. But we never did so, because it didn't occur to us to think of the development list as a resource to be used. We knew that people were probably going to accept whatever we committed, and—because we didn't know what we didn't know—we didn't bother to do the work in a visible way, e.g., posting patches frequently, making small, easily digestible commits to a special branch, etc. The resulting authentication protocol was not very good, and of course, once it became established, it was difficult to improve, because of compatibility concerns.

The root of the problem was not lack of experience; we could easily have learned what we needed to know. The problem was our attitude toward the volunteer development community. We regarded acceptance of the changes as a hurdle to leap, rather than as a process by which the quality of the changes could be improved. Since we were confident that almost anything we did would be accepted (as it was), we made little effort to get others involved.

Obviously, when you're choosing a contractor, you want someone with the right technical skills and experience for the job. But it's also important to choose someone with a track record of constructive interaction with the other developers in the community. That way you're getting more than just a single person; you're getting an agent who will be able to draw on a network of expertise to make sure the work is done in a robust and maintainable way.

# Funding Non-Programming Activities

Programming is only part of the work that goes on in an open source project. From the point of view of the project's volunteers, it's the most visible and glamorous part. This unfortunately means that other activities, such as documentation, formal testing, etc., can sometimes be neglected, at least compared to the amount of attention they often receive in proprietary software. Corporate organizations are sometimes able to make up for this, by devoting some of their internal software development infrastructure to open source projects.

The key to doing this successfully is to translate between the company's internal processes and those of the public development community. Such translation is not effortless: often the two are not a close match, and the differences can only be bridged via human intervention. For example, the company may use a different bug tracker than the public project. Even if they use the same tracking software, the data stored in it will be very different, because the bug-tracking needs of a company are very different from those of a free software community. A piece of information that starts in one tracker may need to be reflected in the other, with confidential portions removed or, in the other direction, added.

The sections that follow are about how to build and maintain such bridges. The end result should be that the open source project runs more smoothly, the community recognizes the company's investment of resources, and yet does not feel that the company is inappropriately steering things toward its own goals.

# Quality Assurance (i.e., Professional Testing)

In proprietary software development, it is normal to have teams of people dedicated solely to quality assurance: bug hunting, performance and scalability testing, interface and documentation checking, etc. As a rule, these activities are not pursued as vigorously by the volunteer community on a free software project. This is partly because it's hard to get volunteer labor for unglamorous work like testing, partly because people tend to assume that having a large user community gives the project good testing coverage, and, in the case of performance and scalability testing, partly because volunteers often don't have access to the necessary hardware resources anyway.

The assumption that having many users is equivalent to having many testers is not entirely baseless. Certainly there's little point assigning testers for basic functionality in common environments: bugs there will quickly be found by users in the natural course of things. But because users are just trying to get work done, they do not consciously set out to explore uncharted edge cases in the program's functionality, and are likely to leave certain classes of bugs unfound. Furthermore, when they discover a bug with an easy workaround, they often silently implement the workaround without bothering to report the bug. Most insidiously, the usage patterns of your customers (the people who drive *your* interest in the software) may differ in statistically significant ways from the usage patterns of the Average User In The Street.

A professional testing team can uncover these sorts of bugs, and can do so as easily with free software as with proprietary software. The challenge is to convey the testing team's results back to the public in a useful form. In-house testing departments usually have their own way of reporting test results, involving company-specific jargon, or specialized knowledge about particular customers and their data sets. Such reports would be inappropriate for the public bug tracker, both because of their form and because of confidentiality concerns. Even if your company's internal bug tracking software were the same as that used by the public project, management might need to make company-specific comments and metadata changes to the issues (for example, to raise an issue's internal priority, or schedule its resolution for a particular customer). Usually such notes are confidential—sometimes they're not even shown to the customer. But even when they're not confidential, they're of no concern to the public project, and therefore the public should not be distracted with them.

Yet the core bug report itself *is* important to the public. In fact, a bug report from your testing department is in some ways more valuable than one received from users at large, since the testing department probes for things that other users won't. Given that you're unlikely to get that particular bug report from any other source, you definitely want to preserve it and make it available to the public project.

To do this, either the QA department can file issues directly in the public issue tracker, if they're comfortable with that, or an intermediary (usually one of the developers) can "translate" the testing department's internal reports into new issues in the public tracker. Translation simply means describing the bug in a way that makes no reference to customer-specific information (the reproduction recipe may use customer data, assuming the customer approves it, of course).

It is somewhat preferable to have the QA department filing issues in the public tracker directly. That gives the public a more direct appreciation of your company's involvement with the project: useful bug reports add to your organization's credibility just as any technical contribution would. It also gives developers a direct line of communication to the testing team. For example, if the internal QA team is monitoring the public issue tracker, a developer can commit a fix for a scalability bug (which the developer may not have the resources to test herself), and then add a note to the issue asking the QA team to see if the fix had the desired effect. Expect a bit of resistance from some of the developers; programmers have a tendency to regard QA as, at best, a necessary evil. The QA team can easily overcome this by finding significant bugs and filing comprehensible reports; on the other hand, if their reports are not at least as good as those coming from the regular user community, then there's no point having them interact directly with the development team.

Either way, once a public issue exists, the original internal issue should simply reference the public issue for technical content. Management and paid developers may continue to annotate the internal issue with company-specific comments as necessary, but use the public issue for information that should be available to everyone.

You should go into this process expecting extra overhead. Maintaining two issues for one bug is, naturally, more work than maintaining one issue. The benefit is that many more coders will see the report and be able to contribute to a solution.

# Legal Advice and Protection

Corporations, for-profit or nonprofit, are almost the only entities that ever pay attention to complex legal

issues in free software. Individual developers often understand the nuances of various open source licenses, but they generally do not have the time or resources to follow copyright, trademark, and patent law in detail. If your company has a legal department, it can help a project by vetting the copyright status of the code, and helping developers understand possible patent and trademark issues. The exact forms this help could take are discussed in Chapter 9, *Licenses, Copyrights, and Patents*. The main thing is to make sure that communications between the legal department and the development community, if they happen at all, happen with a mutual appreciation of the very different universes the parties are coming from. On occasion, these two groups talk past each other, each side assuming domain-specific knowledge that the other does not have. A good strategy is to have a liaison (usually a developer, or else a lawyer with technical expertise) stand in the middle and translate for as long as needed.

# Documentation and Usability

Documentation and usability are both famous weak spots in open source projects, although I think, at least in the case of documentation, that the difference between free and proprietary software is frequently exaggerated. Nevertheless, it is empirically true that much open source software lacks first-class documentation and usability research.

If your organization wants to help fill these gaps for a project, probably the best thing it can do is hire people who are *not* regular developers on the project, but who will be able to interact productively with the developers. Not hiring regular developers is good for two reasons: one, that way you don't take development time away from the project; two, those closest to the software are usually the wrong people to write documentation or investigate usability anyway, because they have trouble seeing the software from an outsider's point of view.

However, it will still be necessary for whoever works on these problems to communicate with the developers. Find people who are technical enough to talk to the coding team, but not so expert in the software that they can't empathize with regular users anymore.

A medium-level user is probably the right person to write good documentation. In fact, after the first edition of this book was published, I received the following email from an open source developer named Dirk Reiners:

```
One comment on Money::Documentation and Usability: when we had some
money to spend and decided that a beginner's tutorial was the most
critical piece that we needed we hired a medium-level user to write it.
He had gone through the induction to the system recently enough to
remember the problems, but he had gotten past them so he knew how to
describe them. That allowed him to write something that needed only
minor fixes by the core developers for the things that he hadn't gotten
right, but still covering the 'obvious' stuff devs would have missed.

His case was even better, as it had been his job to introduce a bunch of
other people (students) to the system, so he combined the experience of
many people, which is something that was just a lucky occurrence and is
probably hard to get in most cases.
```

# Providing Hosting/Bandwidth

For a project that's not using one of the free canned hosting sites (see the section called “Canned Hosting” in Chapter 3, *Technical Infrastructure*), providing a server and network connection—and most importantly, system administration help—can be of significant assistance. Even if this is all your organization does for the project, it can be a moderately effective way to obtain good public relations karma, though it will not bring any influence over the direction of the project.

You can probably expect a banner ad or an acknowledgment on the project's home page, thanking your

company for providing hosting. If you set up the hosting so that the project's web address is under your company's domain name, then you will get some additional association just through the URL. This will cause most users to think of the software as having *something* to do with your company, even if you don't contribute to development at all. The problem is, the developers are aware of this associative tendency too, and may not be very comfortable with having the project in your domain unless you're contributing more resources than just bandwidth. After all, there are a lot of places to host these days. The community may eventually feel that the implied misallocation of credit is not worth the convenience brought by hosting, and take the project elsewhere. So if you want to provide hosting, do so—but either plan to get even more involved soon, or be circumspect about how much involvement you claim.

# Marketing

Although most open source developers would probably hate to admit it, marketing works. A good marketing campaign *can* create buzz around an open source product, even to the point where hardheaded coders find themselves having vaguely positive thoughts about the software for reasons they can't quite put their finger on. It is not my place here to dissect the arms-race dynamics of marketing in general. Any corporation involved in free software will eventually find itself considering how to market themselves, the software, or their relationship to the software. The advice below is about how to avoid common pitfalls in such an effort; see also the section called “Publicity” in Chapter 6, *Communications*.

# Remember That You Are Being Watched

For the sake of keeping the volunteer developer community on your side, it is *very* important not to say anything that isn't demonstrably true. Audit all claims carefully before making them, and give the public the means to check your claims on their own. Independent fact checking is a major part of open source, and it applies to more than just the code.

Naturally no one would advise companies to make unverifiable claims anyway. But with open source activities, there is an unusually high quantity of people with the expertise to verify claims—people who are also likely to have high-bandwidth Internet access and the right social contacts to publicize their findings in a damaging way, should they choose to. When Global Megacorp Chemical Industries pollutes a stream, that's verifiable, but only by trained scientists, who can then be refuted by Global Megacorp's scientists, leaving the public scratching their heads and wondering what to think. On the other hand, your behavior in the open source world is not only visible and recorded; it is also easy for many people to check it independently, come to their own conclusions, and spread those conclusions by word of mouth. These communications networks are already in place; they are the essence of how open source operates, and they can be used to transmit any sort of information. Refutation is usually difficult, if not impossible, especially when what people are saying is true.

For example, it's okay to refer to your organization as having "founded project X" if you really did. But don't refer to yourself as the "makers of X" if most of the code was written by outsiders. Conversely, don't claim to have a deeply involved volunteer developer community if anyone can look at your repository and see that there are few or no code changes coming from outside your organization.

Not too long ago, I saw an announcement by a very well-known computer company, stating that they were releasing an important software package under an open source license. When the initial announcement came out, I took a look at their now-public version control repository and saw that it contained only three revisions. In other words, they had done an initial import of the source code, but hardly anything had happened since then. That in itself was not worrying—they'd just made the announcement, after all. There was no reason to expect a lot of development activity right away.

Some time later, they made another announcement. Here is what it said, with the name and release number replaced by pseudonyms:

> *We are pleased to announce that following rigorous testing by the Singer Community, Singer 5 for Linux and Windows are now ready for production use.*

Curious to know what the community had uncovered in "rigorous testing," I went back to the repository to look at its recent change history. The project was still on revision 3. Apparently, they hadn't found a *single* bug worth fixing before the release! Thinking that the results of the community testing must have been recorded elsewhere, I next examined the bug tracker. There were exactly six open issues, four of which had been open for several months already.

This beggars belief, of course. When testers pound on a large and complex piece of software for any length of time, they will find bugs. Even if the fixes for those bugs don't make it into the upcoming release, one would still expect some version control activity as a result of the testing process, or at least some new issues. Yet to all appearances, nothing had happened between the announcement of the open source license and the first open source release.

The point is not that the company was lying about the community testing. I have no idea if they were or not. But they were oblivious to how much it *looked* like they were lying. Since neither the version control repository nor the issue tracker gave any indication that the alleged rigorous testing had occurred, the company should either not have made the claim in the first place, or provided a clear link to some tangible result of that testing ("We found 278 bugs; click here for details"). The latter would have allowed anyone to get a handle on the level of community activity very quickly. As it was, it only took me a few minutes to determine that whatever this community testing was, it had not left traces in any of the usual places. That's not a lot of effort, and I'm sure I'm not the only one who took the trouble.

Transparency and verifiability are also an important part of accurate crediting, of course. See the section called “Credit” in Chapter 8, *Managing Volunteers* for more on this.

# Don't Bash Competing Open Source Products

Refrain from giving negative opinions about competing open source software. It's perfectly okay to give negative *facts*—that is, easily confirmable assertions of the sort often seen in good comparison charts. But negative characterizations of a less rigorous nature are best avoided, for two reasons. First, they are liable to start flame wars that detract from productive discussion. Second, and more importantly, some of the volunteer developers in *your* project may turn out to work on the competing project as well. This is more likely than it at first might seem: the projects are already in the same domain (that's why they're in competition), and developers with expertise in that domain may make contributions wherever their expertise is applicable. Even when there is no direct developer overlap, it is likely that developers on your project are at least acquainted with developers on related projects. Their ability to maintain constructive personal ties could be hampered by overly negative marketing messages.

Bashing competing closed-source products seems to be more widely accepted in the open source world, especially when those products are made by Microsoft. Personally, I deplore this tendency (though again, there's nothing wrong with straightforward factual comparisons), not merely because it's rude, but also because it's dangerous for a project to start believing its own hype and thereby ignore the ways in which the competition may actually be superior. In general, watch out for the effect that marketing statements can have on your own development community. People may be so excited at being backed by marketing dollars that they lose objectivity about their software's true strengths and weaknesses. It is normal, and even expected, for a company's developers to exhibit a certain detachment toward marketing statements, even in public forums. Clearly, they should not come out and contradict the marketing message directly (unless it's actually wrong, though one hopes that sort of thing would have been caught earlier). But they may poke fun at it from time to time, as a way of bringing the rest of the development community back down to earth.